

Práctica 3: Problema de las 8-Reinas, algoritmos genéticos

Nikhil Baxani Mirchandani

Enrique Pérez Herrera

Gael Espín Borri

Grado en Ciberseguridad e Inteligencia Artificial. Universidad de Málaga.

`nikhilbaxani@uma.es`

`eperezherrera702@uma.es`

`gaeleb14@uma.es`

Este informe trata del diseño, implementación y el análisis de un algoritmo genético para poder resolver el problema de 8-Reinas, en este, vemos el uso de la librería DEAP en python, por el modelamos los cromosomas como permutaciones para satisfacer las restricciones de filas y columnas, también veremos el impacto de los hiperparámetros y la importancia de estos, en la convergencia hacia el fitness deseado, en este caso 28

1 Introducción

El problema de las N -Reinas tiene como objetivo principal posicionar N reinas en un tablero de ajedrez de dimensiones $N \times N$ de tal manera que ninguna de ellas pueda atacar a otra. Según las reglas tradicionales, esto implica que dos piezas no pueden compartir la misma fila, columna o diagonal.

En este proyecto, abordamos el caso de las 8-Reinas ($N = 8$), el espacio de estados y el número de combinaciones posibles crecen de manera drástica, haciendo que los enfoques de búsqueda exhaustiva resulten ineficientes.

Se ha implementado una solución basada en Algoritmos Evolutivos. Esta técnica de búsqueda heurística, inspirada en los principios de la genética biológica y la selección natural, permite explorar el espacio de soluciones de forma inteligente y dirigida. Mediante la aplicación iterativa de operadores de selección, cruce y mutación sobre una población de cromosomas, el algoritmo es capaz de converger hacia el óptimo global evitando estancarse en óptimos locales.

El desarrollo práctico de este trabajo se ha llevado a cabo utilizando el lenguaje de programación Python, apoyándose en la librería DEAP (*Distributed Evolutionary Algorithms in Python*)

2 Modelado del Problema

Para aplicar este algoritmo genético en N -reinas, debemos formalmente definir como se representan las soluciones, y cómo se evalúa la calidad de estas, los detalles del modelo son los siguientes:

2.1 Definición del Individuo

Lenguaje Natural: Para optimizar la búsqueda y garantizar que nunca haya dos reinas en la misma fila ni en la misma columna desde el inicio, podríamos usar una matriz de 64, sin embargo no es nada eficiente, por eso, decidimos montar un vector de 8, en el que cada posición de la fila indica donde está situada la reina. Para cumplir con las restricciones del ajedrez, el individuo es siempre una permutación sin repetición de los números del 1 al 8.

Lenguaje Matemático: Un individuo se define como un vector $X = (x_1, x_2, \dots, x_n)$ donde $n = 8$. Cada $\text{gen}(\text{generacion}, \text{iteracion})$ representa una fila $x_i \in \{1, 2, \dots, n\}$. Dado que X es una permutación, se cumple la restricción de que $x_i \neq x_j$ para todo $i \neq j$.

2.2 Definición de la Población

Lenguaje Natural: La población es el conjunto de individuos (tableros candidatos) que coexisten y evolucionan en una misma iteración o generación del algoritmo.

Lenguaje Matemático: Una población en la generación t es un conjunto discreto $P(t) = \{X_1, X_2, \dots, X_M\}$, donde M representa el tamaño total de la población.

2.3 Función de Aptitud (Fitness)

Lenguaje Natural: El problema se ha modelado como una función de maximización (llegar al mayor fitness posible). La aptitud de un individuo equivale al número total de parejas de reinas en el tablero que no se atacan entre sí. Puesto que la representación del cromosoma ya evita los ataques ortogonales (filas y columnas), la función únicamente penaliza los ataques en diagonal. Para $n = 8$, el número máximo de parejas posibles es 28, siendo este el valor óptimo a alcanzar.

Lenguaje Matemático: La función a maximizar se define como:

$$f(X) = 28 - \sum_{i=1}^{n-1} \sum_{j=i+1}^n A(x_i, x_j) \quad (1)$$

Donde la función de penalización por ataque diagonal $A(x_i, x_j)$ es:

$$A(x_i, x_j) = \begin{cases} 1 & \text{si } |i - j| = |x_i - x_j| \\ 0 & \text{en caso contrario} \end{cases} \quad (2)$$

3 Configuración del Algoritmo Genético

La evolución de la población se rige por tres operadores genéticos principales, implementados a través de la librería DEAP:

- **Cruce (Crossover):** Se ha empleado el cruce *Partially Matched Crossover* (PMX). Este operador es fundamental al trabajar con permutaciones, ya que permite intercambiar secuencias genéticas entre dos padres asegurando que los hijos resultantes no contengan valores repetidos (evitando así violar las restricciones de fila).
- **Mutación:** Se utiliza el operador *Shuffle Indexes*, el cual selecciona aleatoriamente dos genes (reinas) dentro del cromosoma y permuta sus posiciones. Esto introduce variabilidad genética y ayuda a escapar de óptimos locales.
- **Selección:** Se aplica un método de selección por *Torneo* (con tamaño de torneo de 3). Esto hace que se seleccionen aleatoriamente a tres individuos de la población y permite que el de mayor fitness pase a la siguiente generación, manteniendo una presión selectiva adecuada sin perder diversidad de manera equivocada.

4 Resultados y Análisis de Rendimiento

Para la resolución mediante el algoritmo genético, y comprender cómo influyen los hiperparámetros en la convergencia para solución óptima, hemos demostrado diferentes ejemplos, cabe destacar que hemos partido de una semilla para poder replicar el experimento siendo esta (`random.seed(42)`).

4.1 Escenario Base (Configuración Óptima)

En este primer experimento, se ha utilizado la configuración estándar: un tamaño de población de 100 individuos, una probabilidad de cruce del 80% ($P_c = 0.8$) y una probabilidad de mutación del 20% ($P_m = 0.2$), evolucionando durante 50 generaciones.

Como se puede observar en la Figura 1, el fitness maximo llega en la 5 generacion de manera rapida al max de 28 demostrando la eficacia de la representación elegida.

Sin embargo, se aprecia que el máximo generacional baja posteriormente, cayendo a 27, volviendo a alcanzar 28 de forma puntual, y finalmente estabilizándose en 27 a partir de la generación 18. aunque se descubren soluciones perfectas, las altas tasas de cruce y de mutación alteran el material genético de estos individuos superiores en las generaciones posteriores. Con Hall of fame, mantenemos la mejor generación

El fitness medio (línea azul punteada) muestra una curva de aprendizaje saludable, ascendiendo progresivamente desde un valor inicial de 23 hasta converger cerca del 26.8, indicando que la calidad global de la población mejora iterativamente.

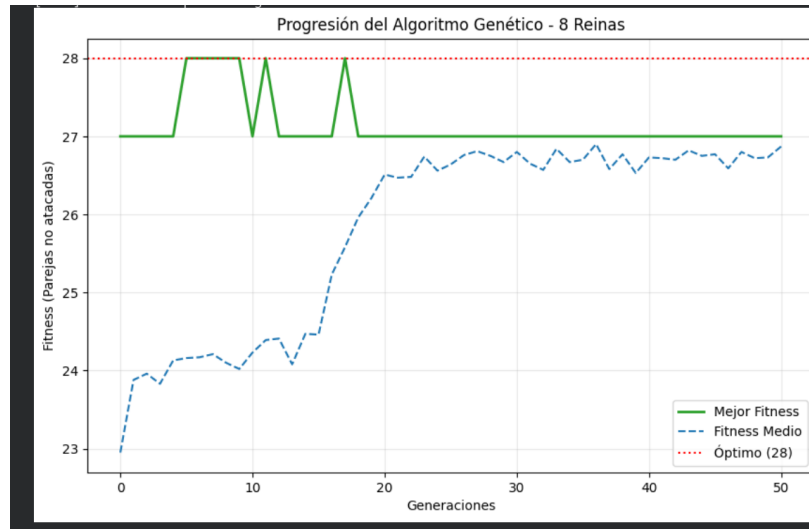


Fig. 1. Progresión del fitness en el Escenario Base (Pob=100, Cruce=0.8, Mut=0.2).

4.2 Experimento 2: Impacto de la Baja Mutación

Bajamos la probabilidad de mutación. Como se puede observar en la Figura 2, el comportamiento cambia bastante respecto al caso anterior(la hemos bajado la probabilidad a 0.01).

El fitness máximo (línea verde) vuelve a encontrar el 28 rapidísimo. Aunque tiene un par de caídas a 27 al principio, a partir de la generación 20 se clava en el máximo de 28 y ya no se mueve en toda la ejecución.

El fitness medio Empieza en 23 y va subiendo, pero a partir de la generación 30 pega un salto y prácticamente se pega a la línea verde del máximo.

Al bajar la mutación, quitamos parte de mutación de caos factor de "caos". Una vez que el algoritmo encuentra la solución perfecta

4.3 Experimento 3: Impacto de una Población Pequeña

Para este último experimento, redujimos drásticamente el tamaño de la población a solo 10 individuos.

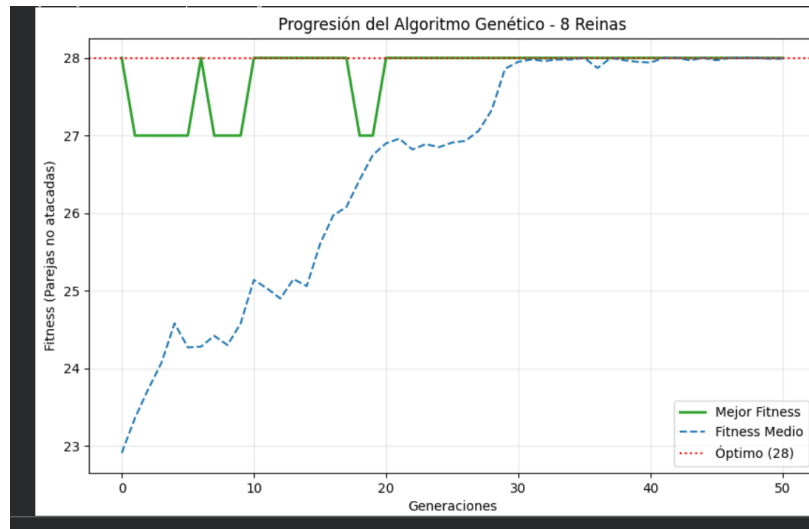


Fig. 2. Progresión del fitness con convergencia total de la población, con bajada de probabilidad de mutación.

El fitness máximo (línea verde) arranca en 26 y se queda atascado ahí prácticamente toda la ejecución. Aunque tiene un pequeño pico de "suerte" alrededor de la generación 7 donde logra tocar el 27, pierde esa configuración rápidamente, vuelve a caer a 26 y se queda completamente plano hasta el final.

El fitness medio vemos sube y baja de 25 y 26, al tener una población de solo 10 individuos, cualquier individuo nuevo que salga muy malo por culpa de una mutación o un cruce fallido va a arrastrar hacia abajo la nota media de toda la generación.

En resumen, esta gráfica demuestra perfectamente la importancia de la diversidad genética inicial.

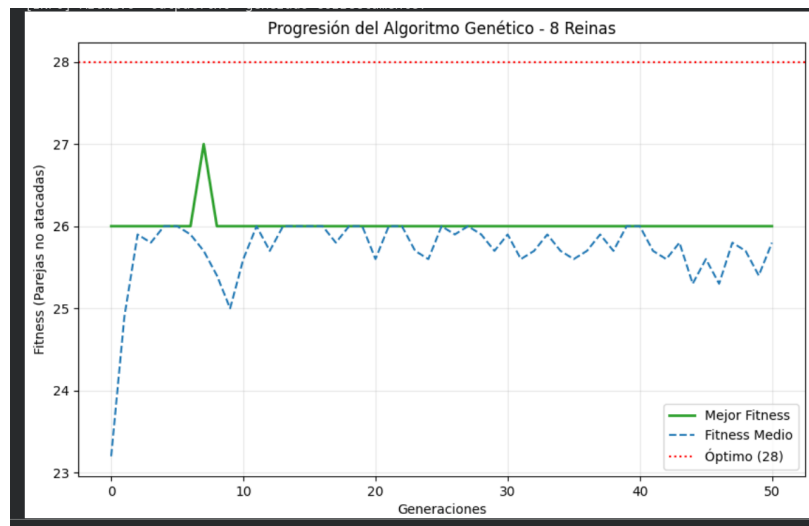


Fig. 3. Estancamiento y alta volatilidad por falta de diversidad (Población = 10).

5 Conclusiones

En este proyecto se ha logrado resolver con éxito el clásico problema de las 8-Reinas mediante la implementación de un algoritmo genético. El objetivo era colocar 8 reinas en un tablero de ajedrez sin que se atacaran entre sí, y nuestro modelo ha sido capaz de encontrar la solución perfecta (alcanzando el fitness máximo de 28 parejas seguras) de manera rápida y eficiente.

También hemos aprendido la importancia de elegir los hiperparámetros óptimos y necesarios para poder realizar el problema, que pueden afectar ya sea la probabilidad de mutación o de cruce, y también la población de cada generación

6 Uso de IA

Se empleó Gemini para la estructuración del informe en formato LaTeX y ayuda a la generación del código necesario para esta práctica.

Declaración de Originalidad

Asumimos la originalidad del trabajo. El código y análisis son de elaboración propia para fines académicos.